

RETRIEVAL ENGINEERING · APPLIED RAG

Choosing a Chunking Strategy

A decision framework for RAG systems: how to pick a chunking strategy from document shape, query shape and cost budget — instead of copying the default from a tutorial.

RAG

VECTOR SEARCH

CONTEXT ENGINEERING

RETRIEVAL EVAL

DECISION GUIDE

9

STRATEGIES
MAPPED

5

QUESTIONS IN THE
DECISION PATH

16

CORPUS SCENARIOS
WITH DEFAULTS

1

EVAL LOOP THAT
SETTLES ARGUMENTS

What this document decides for you

Chunking is the highest-leverage, least-examined parameter in most RAG stacks. This guide replaces "512 tokens with 50 overlap because that is what the tutorial said" with a defensible decision, a scenario default, and a way to prove it.

SEC 01 Chunking is a retrieval decision, not preprocessing

The two decisions everyone conflates: boundary policy and assembly policy. The three forces that govern both.

SEC 02 The nine strategies, honestly rated

Five boundary policies, four assembly layers — index cost, best fit, and the exact failure that kills each one.

SEC 03 The five-question decision path

Structure → answer shape → chunk independence → budget and churn → citation and access control. Run it in order.

SEC 04 The scenario playbook

Sixteen corpus types with a starting configuration for each: boundary, size, overlap, assembly layer, and why.

SEC 05 Size, overlap and the metadata that must ride along

Tuning heuristics tied to your embedding model, plus the chunk envelope that makes citation and re-indexing possible.

SEC 06 Eight anti-patterns that quietly destroy recall

The failures that never raise an exception — they just return the wrong chunk with high confidence.

SEC 07 How to actually decide: the eval loop

A golden set, five metrics, a 3×2 test matrix, and a priority ordering for what to fix first.

SEC 08 One-page cheat sheet and rollout plan

The compressed if-then table, a three-sprint adoption path, and where to go next.

READER'S SHORTCUT

If you need an answer in sixty seconds: recursive structure-aware splitting at **400–800 tokens with 10–15% overlap**, hybrid dense + BM25 retrieval, and a reranker over the top 20 is the right default for roughly 80% of corpora. Sections 03 and 04 tell you whether you are in the other 20%. Section 07 tells you how to prove it either way.

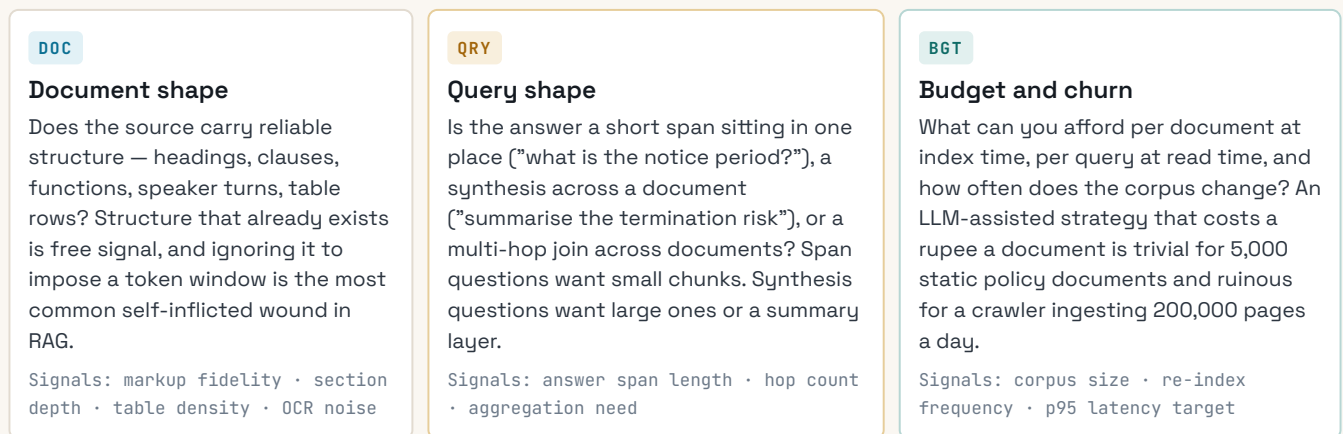
Chunking is a retrieval decision, not preprocessing

The chunk is doing two jobs at once — it is the unit that gets matched and the unit that gets read. Most chunking arguments are really arguments about whether those two jobs should be done by the same object.



Two of those six stages are chunking decisions, and they are independent. **Boundary policy** decides where the text is cut and therefore what a vector represents. **Assembly policy** decides what actually reaches the model once a chunk has matched — the chunk alone, its parent section, its neighbours, or a rewritten version of it. Teams that treat chunking as a single knob spend weeks tuning chunk size when the real fix was to retrieve small and read big.

The three forces



What the evidence actually supports



Three published results shape the whole decision. First, **context loss hurts more than boundary placement**: Anthropic’s contextual retrieval work found that prepending a short generated description of where a chunk sits in its document cut top-20 retrieval failures by 35% on its own, 49% with contextual BM25, and 67% with reranking added. Second, **clever boundaries are oversold**: the NAACL 2025 study by Qu, Tu and Bao found that semantic chunking’s computational cost is not justified by consistent gains over plain fixed-size splitting. Third, **ordering beats splitting**: late chunking — encoding the whole document first and pooling per-chunk vectors afterwards — improved retrieval across every boundary strategy it was paired with, which means it is a layer on top of your boundary choice rather than a replacement for it.

THE UNCOMFORTABLE IMPLICATION

If you are choosing between semantic chunking and recursive splitting before you have added hybrid search, a reranker, or document context to your chunks, you are optimising the parameter with the smallest published effect size. Do the cheap high-yield work first, then tune boundaries against your own eval set.

The one rule that survives every corpus

- **A chunk must be interpretable alone.** If a human analyst handed that chunk on an index card could not tell you what document, section and entity it refers to, the embedding cannot either.
- **A chunk must be smaller than the answer's blast radius.** Retrieval precision collapses when one vector has to represent five unrelated ideas.
- **A chunk must be reconstructable.** Stable IDs, character offsets and a section path, or you can never cite it, diff it or re-index it safely.

The same clause, three ways

A single worked example makes the boundary-versus-assembly distinction concrete. Take one clause from a master services agreement:

```
SOURCE  "7.3 Termination. Either party may terminate this Agreement on sixty (60)
         days' written notice. This right shall not apply during the Initial Period."

A. FIXED WINDOW (256 tok)  → chunk ends after sentence two; the exception becomes
                           "This right shall not apply during the Initial Period."
                           Embeds as: an unattributed sentence about an unnamed right.

B. STRUCTURE-AWARE (clause) → the whole of 7.3 stays together.
                           Matches "notice period", "how do we terminate".
                           Still misses "how do we exit the Vertex contract".

C. B + CONTEXTUAL PREFIX  → "Clause 7.3 of the Acme-Vertex Master Services
                           Agreement (2026), setting out termination rights."
                           Matches the entity question and the clause question.
```

Configuration A fails on **boundary** — the exception was severed from the right it modifies, and no reranker can repair a vector that never encoded the subject. Configuration B fixes the boundary but still fails a query phrased in the vocabulary of the contract rather than the clause, because the party names appear only on page one. C fixes that with an assembly-layer change and no boundary change at all. Two different failures, two different levers — which is exactly why the questions in Section 03 are asked in that order.

The nine strategies, honestly rated

Five ways to place boundaries, four ways to assemble what gets read. Mixing one from each column is normal and usually correct — they are not competing options.

COLUMN A — BOUNDARY POLICIES (WHERE YOU CUT)

FIX

Fixed-size window

N tokens with M overlap, ignoring content. Fast, deterministic, trivially parallel, and a far stronger baseline than its reputation suggests.

Cost: negligible · Fit: homogeneous prose, high-churn crawls

REC

Recursive separator split

Try paragraph, then sentence, then word boundaries until the chunk fits the size budget. The sane default: fixed-size predictability without mid-sentence cuts.

Cost: negligible · Fit: almost everything, as a starting point

STR

Structure-aware split

Cut on the document's own seams — Markdown headings, HTML sections, contract clauses, code functions via AST, speaker turns, table rows. Highest yield per engineering hour when the structure is real.

Cost: low (parser work) · Fit: docs, contracts, code, transcripts

SEM

Semantic breakpoint

Embed sentences, cut where consecutive similarity drops past a percentile threshold. Intuitive and demo-friendly; published evaluations show inconsistent gains for a real embedding bill at index time.

Cost: 1 embedding pass per sentence · Fit: unstructured prose with topic drift

LLM

LLM / proposition chunking

A model decides boundaries or rewrites passages into standalone propositions. Best quality on messy, high-value corpora; the only strategy whose cost scales with a token bill rather than CPU.

Cost: high per document · Fit: small, static, high-stakes corpora

COLUMN B — ASSEMBLY LAYERS (WHAT ACTUALLY GETS READ)

P2B

Parent-document / small-to-big

Index small precise chunks, but return the parent section or a window of neighbours to the model. Decouples match granularity from read granularity — the single highest-value trick in this document.

Cost: storage only · Adds: precision without context starvation

CTX

Contextual retrieval

Prepend a 50–100 token generated line situating each chunk in its document before embedding and BM25 indexing. Attacks context loss directly and is the best-evidenced upgrade available.

Cost: one cached LLM pass per doc · Adds: up to 49% fewer failed retrievals with BM25

LATE

Late chunking

Encode the full document through a long-context embedding model first, then mean-pool token spans into per-chunk vectors. Pronouns and references inherit meaning from the whole document.

Cost: long-context embedding call · Adds: small consistent gain on any boundary policy

HIER

Hierarchical summary index

Build a tree: chunk → section summary → document summary, and retrieve at whichever level the query needs. The answer for "summarise", "compare across", and other questions no single chunk can satisfy.

Cost: summarisation pass per node · Fit: synthesis and multi-hop workloads

NONE

No chunking (full document)

Long-context models make "just send the document" viable for small corpora. Retrieval quality is perfect; cost, latency and lost-in-the-middle effects are the price. Legitimate below roughly 50 documents.

Cost: per-query tokens · Fit: tiny corpora, one-off analysis

STRATEGY	INDEX COST	CHOOSE IT WHEN	IT BREAKS WHEN	VERDICT
Fixed-size	None	The corpus is huge, homogeneous and re-indexed constantly; you need a baseline to beat.	Documents contain tables, code or clause structure that the window slices through.	BASELINE
Recursive	None	You want fixed-size behaviour without mid-sentence cuts. The default first build.	Separators are meaningless — OCR soup, minified HTML, single-paragraph transcripts.	ADOPT
Structure-aware	Low	The source has headings, clauses, functions or turns you can trust. Almost always worth the parser.	Structure is inconsistent across sources, or one section is 40 pages long and still needs sub-splitting.	ADOPT
Semantic	Medium	Prose has genuine topic drift with no markup — long-form interviews, unstructured reports.	You adopt it on faith; published evaluations find gains inconsistent versus fixed-size for the added cost.	PROVE IT
LLM / proposition	High	Small, static, high-value corpus where a wrong retrieval has real consequences.	The corpus grows or churns — cost scales linearly and re-chunking becomes a budget item.	SELECTIVE
Parent-document	Storage	Precise matching matters but the model needs surrounding context to answer well. Nearly universal.	Parents are so large they blow the generation budget — cap parent size explicitly.	ADOPT
Contextual retrieval	Low w/ caching	Chunks reference entities defined elsewhere in the document — filings, manuals, codebases.	Generated context is a near-paraphrase of the chunk, adding tokens without adding signal.	ADOPT
Late chunking	Low	Your embedding model supports it and documents fit its context window. Free-ish accuracy.	Documents far exceed the encoder's window, or your provider does not expose the parameter.	LAYER ON
Hierarchical index	Medium	Users ask synthesis, comparison and "what changed" questions no single chunk answers.	Summaries drift from source and become an unauditable second source of truth.	CONDITIONAL

Three stacks that cover most builds

The starter stack — recursive boundaries at 500 tokens with 12% overlap, hybrid dense and BM25 retrieval, reranking over the top 20. Build this first regardless of corpus; it is the baseline every later change is measured against. **The document stack** — structure-aware boundaries on the native seam, parent-document assembly, contextual prefixes carrying entity and version. This is the right shape for contracts, filings, manuals and technical documentation, and it is where most enterprise deployments should end up. **The high-stakes stack** — the document stack plus proposition rewriting and a hierarchical summary level, justified only when the corpus is small enough to re-chunk on demand and a wrong retrieval carries real consequences.

What you should not do is assemble a fourth stack out of whichever techniques were most recently written about. Every layer added here costs index time, query latency, or both, and each one needs a measured reason to exist — which is what Section 07 is for.

The five-question decision path

Run these in order. Each question eliminates options; by question five you have a boundary policy, a size band and an assembly layer. Stop at the first question that gives a decisive answer for your corpus.

Q1 • STRUCTURE

Does the document carry structure you can trust?

Not "does it have headings somewhere" — does every document in the corpus have them, parsed reliably, at a usable granularity? Test on 20 random documents before committing.

YES → Structure-aware split on the native seam (heading, clause, function, turn), with a size cap that sub-splits oversized sections recursively.

NO → Recursive splitting. Consider semantic boundaries only if you will measure them.



Q2 • ANSWER SHAPE

Is the answer a span, a section, or a synthesis?

Pull 30 real questions from your users or support logs and mark where the answer physically lives. This single exercise resolves most chunk-size debates without a benchmark.

SPAN (a sentence or two) → Small chunks, 200-400 tokens, plus parent-document assembly.

SECTION → Structure-aware chunks at 600-1,000 tokens, returned whole.

SYNTHESIS / MULTI-HOP → Add a hierarchical summary level; chunk size alone will not fix this.



Q3 • INDEPENDENCE

Can a chunk be understood without its neighbours?

Print ten random chunks. If they open with "it", "this section", "the above requirement", or a bare number whose subject sits three pages earlier, your embeddings are encoding ambiguity.

NO → Add contextual retrieval (generated prefix + contextual BM25), or late chunking if your embedding provider supports it. Both, if the corpus justifies it.

YES → Skip the augmentation layer and spend the budget on reranking instead.



Q4 • BUDGET & CHURN

What does one document cost to index, and how often does it change?

Multiply your per-document strategy cost by corpus size, then again by expected re-index cycles per year. Any LLM-in-the-loop strategy must survive that second multiplication.

HIGH CHURN / LARGE CORPUS → Deterministic boundaries only; keep LLM passes for a curated high-value subset.

STATIC / SMALL → LLM or proposition chunking is affordable and usually wins on quality.



Q5 • CITATION & ACL

Do you have to cite, audit or restrict what was retrieved?

Regulated domains make this non-negotiable, and it constrains boundaries: a chunk that spans two clauses or two tenants cannot be cited or filtered cleanly.

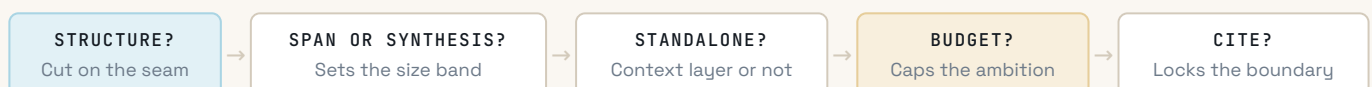
YES → Never cross a structural or permission boundary in a chunk. Carry `doc_id`, section path, page, char offsets, version and ACL scope on every chunk. Prefer smaller chunks with parent expansion over large merged ones.

NO → Optimise purely for recall and cost.

IF YOU ANSWERED "I DON'T KNOW" TO THREE OR MORE

Ship the default — recursive, structure-aware where headings exist, 500 tokens, 12% overlap, hybrid retrieval, rerank top 20 — and build the eval set from Section 07 this sprint. You will learn more from 60 labelled queries against a mediocre baseline than from another week of choosing.

The decision compressed



The scenario playbook

Sixteen corpus types with a starting configuration. These are defensible starting points, not final answers — every one of them should be re-tested against the eval loop in Section 07 once you have real queries.

DOCUMENTS & KNOWLEDGE BASES

SCENARIO	BOUNDARY	SIZE / OVERLAP	ASSEMBLY	WHY THIS CONFIGURATION
Legal contracts & agreements	Structure-aware: clause and sub-clause	Clause-bounded, 200–600 tok, no overlap	Parent = full section; defined-terms glossary injected	Clauses are the legal unit of meaning and the unit of citation. Overlap corrupts clause identity; a definitions lookup solves the "capitalised term defined on page 2" problem better than bigger chunks.
Policy & compliance manuals	Structure-aware: heading hierarchy	400–800 tok, 10%	Contextual prefix with policy name, version, effective date	Near-identical wording recurs across versions and jurisdictions; without document-level context the retriever cannot tell the 2024 policy from the 2026 one.
API & product documentation	Structure-aware: h2 / h3 , keep code block with its prose	300–700 tok, 0–10%	Parent = full page section	A snippet without its explanation, or an explanation without its snippet, produces confidently wrong answers. The heading path is also excellent BM25 signal.
Internal wiki / Confluence	Recursive + heading aware	500 tok, 12%	Contextual prefix (space, owner, last-updated)	Wikis are structurally inconsistent and full of stale duplicates; freshness and ownership metadata do more for answer quality than boundary tuning.
Research papers (PDF)	Structure-aware: section, with figures and tables extracted separately	600–1,000 tok, 10%	Hierarchical: abstract + section summaries above chunks	Methods and results answer different questions; abstracts serve the "what is this paper about" query far better than any body chunk.
Financial filings (10-K, annual reports)	Structure-aware: item/section; tables kept intact	500–800 tok, 10%	Contextual prefix: company, fiscal period, statement type	The canonical context-loss case — a revenue figure means nothing without the entity and the quarter attached to it. This is exactly where contextual chunking shows its largest measured gains.

CODE, CONVERSATION & OPERATIONAL DATA

SCENARIO	BOUNDARY	SIZE / OVERLAP	ASSEMBLY	WHY THIS CONFIGURATION
Source code repositories	AST: function / class, imports header carried	Function-bounded, no overlap	Parent = file; contextual prefix with file path and module role	Functions are the semantic unit; a mid-function cut is worthless. Path and imports are how developers actually search, so they belong in the indexed text.
Support tickets & CRM notes	One ticket = one chunk (split only if very long)	Whole record, ≤800 tok	Metadata filters: product, severity, resolution status	The problem-and-resolution pair is the atomic unit of value. Splitting it separates the symptom from the fix — the exact thing the user is asking for.
Meeting & call transcripts	Speaker turn, merged into topic windows	60–90 seconds of speech, 1 turn overlap	Contextual prefix: meeting, date, participants	Turns are short and pronoun-heavy; merging into topical windows restores meaning, and the prefix disambiguates the twelve standups that all mention "the migration".

SCENARIO	BOUNDARY	SIZE / OVERLAP	ASSEMBLY	WHY THIS CONFIGURATION
Chat & community logs	Thread or time-gap session boundary	Session-bounded, ≤600 tok	Parent = full thread	Conversation drifts continuously; time gaps and reply chains are far better boundary signals than token counts or embedding distance.
Clinical & patient notes	Structure-aware: note section (HPI, assessment, plan)	Section-bounded, no overlap	Strict ACL and encounter-date metadata; parent = encounter	Sections carry different clinical meaning and different sensitivity. Never let a chunk straddle two patients or two encounters — that is a safety and privacy boundary, not a retrieval one.
Product catalogues & structured records	One record = one chunk, rendered as a natural-language template	100–300 tok	Attribute filters pushed to metadata, not embeddings	Rows are already atomic. Numeric and categorical constraints belong in filters; embeddings should carry only the descriptive text.
News & web crawl at scale	Recursive, boilerplate stripped first	400–600 tok, 10%	Recency-weighted ranking; late chunking if supported	Volume and churn rule out per-document LLM passes. The cheapest deterministic boundary plus good filtering beats sophisticated chunking you cannot afford to re-run.
Slide decks & scanned PDFs	Page or slide, with layout-aware OCR	Page-bounded	Contextual prefix: deck title, section, slide number; VLM caption for figures	Reading order is unreliable after OCR, which makes token windows dangerous. The page is the only boundary you can trust; captions rescue the content trapped in images.
Spreadsheets & CSV exports	Row group, header row repeated in each chunk	10–50 rows	Prefer SQL or text-to-query over vector search where possible	Aggregation questions (“total across regions”) are not retrieval questions. Vector search over tabular data is a fallback, not a first choice.
Mixed enterprise corpus	Router: per-source-type boundary policy	Varies by type	Per-type index, single federated retriever	One chunk size across contracts, code and tickets guarantees mediocrity in all three. Route by source type at ingestion — it is a day of work and the largest single win in most enterprise deployments.

HOW TO USE THIS TABLE

Take the row that matches your dominant source type and ship it this week — these configurations are chosen to be safe rather than optimal. Then read the “why” column carefully; it names the specific failure each choice is defending against. If that failure does not exist in your corpus, the configuration is over-engineered for you; if a failure you are seeing is not named anywhere in the column, Section 06 probably has it.

Two patterns repeat across all sixteen rows and are worth stating plainly. **The boundary is almost always something the document already has** — a clause, a function, a turn, a page, a row — and the engineering work is extraction, not invention. **The assembly layer is almost always about identity** — which entity, which version, which period, which tenant this text belongs to. Boundaries make chunks readable; assembly makes them findable.

Size, overlap and the metadata that must ride along

Once the strategy is chosen, three parameters remain. Two of them are widely over-thought; the third — the chunk envelope — is widely ignored and causes most production pain.

Chunk size

Size is bounded from below by the length of a typical answer span and from above by your embedding model's maximum sequence length. Anything longer is silently truncated by most encoders — the API returns a vector, no error is raised, and the tail of your chunk is simply not represented. Verify the limit for your specific model and count tokens with that model's tokenizer, not a character heuristic and not a different vendor's.

BAND	USE WHEN	STRENGTH	COST OF CHOOSING IT
128–256 tokens	FAQ, catalogue records, span-level lookup	Sharpest embeddings; excellent precision on short factual queries	Context starvation unless paired with parent expansion; large index, more vectors to rerank
400–800 tokens	The default for prose, documentation, policies	Best measured balance of precision and self-containment across most corpora	None material — start here unless you have a reason not to
1,000–1,500 tokens	Narrative documents, reports read section by section	Chunks answer questions without expansion; fewer vectors	Diluted embeddings — one vector representing several ideas retrieves for none of them well
Whole document	Corpora under roughly 50 documents	Zero retrieval error; the model sees everything	Latency, token cost, and degraded attention to material in the middle of long contexts

Overlap

Overlap exists to stop an answer being severed at a boundary. **10–15% is the working range.** Below 5% you sever answers; above 25% you are paying to store and rerank near-duplicates, and your top-k fills with three copies of the same passage — which is worse than it sounds, because it crowds out the genuinely different chunk that held the rest of the answer. If you have adopted structure-aware boundaries, set overlap to zero: the boundary is already meaningful, and overlap only blurs it. Deduplicate at retrieval time regardless.

The chunk envelope

Every chunk should be stored with the fields below. This is not bookkeeping — it is what makes citation, permission filtering, incremental re-indexing and A/B comparison possible. Retrofitting it after launch means re-indexing the whole corpus.

```
{
  "chunk_id":      "sha256(doc_id + version + char_start)",
  "doc_id":        "stable across versions",
  "doc_version":   "content hash or CMS revision",
  "section_path":  ["Part II", "Clause 7", "7.3 Termination"],
  "page":          14,
  "char_start":    18422,
  "char_end":      19105,
  "parent_id":     "id of the section chunk to expand to",
  "acl_scope":     ["tenant:acme", "role:legal"],
  "source_type":   "contract | code | ticket | transcript",
  "chunk_config":  "recursive-v3:800:0.12",
  "indexed_at":    "06/09/2026T11:20:00+05:30"
}
```

- **chunk_config is the field teams forget.** Stamping the strategy and parameters on every chunk is what lets you run two configurations side by side, compare them honestly, and roll back a bad re-chunk without rebuilding the index.

- **Offsets make citation real.** Character ranges let the UI highlight the exact source span instead of showing an approximate paragraph — the difference between a system users trust and one they double-check.
- **ACL scope belongs on the chunk, not the document.** Filtering at query time is the only reliable way to keep a retriever from leaking across tenants or roles.

Two special cases worth codifying

TBL

Tables

Never split a table across chunks, and repeat the header row in every table chunk. Attach a one-line generated caption stating what the table shows and which entity and period it covers — a bare grid of numbers embeds poorly and retrieves worse.

CODE

Code blocks

Split on AST nodes, never on line counts. Carry the file path and relevant imports as a header on each chunk, and keep a function's docstring attached to its body — the docstring is usually what the query actually matches.

Size is a property of the pair, not the corpus

The optimal chunk size belongs to the combination of your corpus and your embedding model, which is why borrowed numbers travel so badly. A model trained on short retrieval passages behaves differently at 900 tokens than one built for long documents, and models differ in how gracefully quality degrades as a chunk approaches their sequence limit. Treat every embedding model swap as an invalidation event: the same eval set, re-run, before the migration is called done.

THREE ASSERTIONS WORTH ADDING TO THE INGEST JOB

One: every chunk's token count, measured with the embedding model's own tokenizer, is under the model limit — fail the batch, do not silently truncate. **Two:** no chunk is under about 50 tokens; fragments that short are usually orphaned headings or table debris that pollute the top-k. **Three:** every chunk carries a non-null `parent_id` and `section_path`, so nothing enters the index that cannot be expanded or cited later.

Eight anti-patterns that quietly destroy recall

None of these raise an exception. The pipeline runs, the dashboard is green, and the system returns a plausible wrong chunk with a high similarity score.

01

Silent truncation past the encoder limit

Chunks longer than the embedding model's maximum sequence length are cut without warning. Half your chunk is unsearchable and nothing tells you. Assert on token length at index time.

02

Orphan references

"It must be replaced after 10,000 hours" retrieves for nothing, because the pump it refers to is in the previous chunk. This is the single most common cause of unexplained recall failures — and the reason contextual retrieval and late chunking exist.

03

One chunk size for a heterogeneous corpus

Contracts, changelogs and transcripts share nothing structurally. A single global setting is a decision to serve all three badly. Route by source type instead.

04

Overlap bloat

Overlap above 25% fills the top-k with near-duplicates, wastes the generation budget, and hides the second half of the answer below the cut-off. More overlap is not more safety.

05

Mid-table and mid-code splits

A table severed from its header, or a function severed mid-body, produces chunks that are individually meaningless and collectively misleading. Detect these structures before splitting, not after.

06

Adopting semantic chunking on faith

It demos beautifully and adds a real embedding bill. Published evaluation finds the gains inconsistent against plain fixed-size splitting. Fine to use — not fine to use unmeasured.

07

Re-chunking without versioning

Changing the strategy while old vectors remain leaves an index containing two incompatible representations. Every quality metric becomes noise. Stamp the config, re-index atomically, keep the old index until the new one wins.

08

Tuning chunking before fixing retrieval

Hybrid dense + BM25 search and a reranker typically deliver larger gains than any boundary change, for less work. If those are missing, chunk-size experiments are measuring the wrong variable.

DIAGNOSTIC THAT TAKES TEN MINUTES

Sample 20 chunks at random and read them as a stranger would. Count how many you could not attribute to a document, a section and a subject. That percentage is the ceiling on your retrieval quality, and no amount of reranking will lift it — the information is not in the vector.

How to actually decide: the eval loop

Every recommendation in this document is a prior. The eval set is what turns it into a decision — and it is cheaper to build than one week of arguing about chunk size.



Build the golden set

Fifty to a hundred real user questions, each labelled with the passage that actually answers it. Draw them from support logs, sales calls or a pilot group — not from a model asked to invent questions, which produces queries suspiciously well matched to the chunks you already have. Include the hard cases deliberately: multi-hop questions, questions whose answer spans a boundary, questions with no answer in the corpus at all. Roughly 20% unanswerable is a healthy proportion, because it is the only way to measure whether your system knows when to say it does not know.

Measure five things

METRIC	WHAT IT TELLS YOU	ACTS ON
Recall@k (k = 20)	Whether the answer was retrievable at all. The primary chunking metric — everything downstream is capped by it.	Boundary policy, context augmentation
Context precision	How much of what you sent was relevant. Low precision means chunks too large or overlap too high.	Chunk size, overlap, deduplication
Answer faithfulness	Whether the generated answer is grounded in the retrieved text. Falls when chunks are severed mid-argument.	Assembly layer, parent expansion
Index cost / 1k docs	What the strategy costs to build and rebuild. The number that kills LLM chunking at scale.	Strategy selection under churn
p95 query latency	What the user experiences. Reranking and hierarchical expansion both spend here.	Top-k, rerank depth, expansion size

Run a 3 × 2 matrix, not a grid search

Three boundary configurations — your current one, a structure-aware variant, and one size band away from your default — crossed with two assembly layers: plain chunks, and chunks plus contextual prefixes or parent expansion. Six runs answers the question. Exhaustive sweeps over size and overlap produce differences smaller than the noise in a hundred-query eval set, and consume the time that should have gone into labelling more queries.

Fix in this order

PRIORITY	MOVE	RATIONALE
★ CORE	Hybrid dense + BM25 retrieval, reranker over top 20	Largest measured gain per unit of effort, independent of any chunking decision. Do this before touching boundaries.
★ CORE	Structure-aware boundaries where structure exists	Costs a parser and pays back permanently across every query type.
SUPP	Parent-document assembly	Decouples match precision from read context; storage is the only meaningful cost.
SUPP	Contextual prefixes on chunks	Best evidence base of any augmentation, and prompt caching keeps the per-document cost trivial.
HABIT	Assert token length; stamp <code>chunk_config</code>	Two lines of code that prevent the two most expensive silent failures in this document.
PERIODIC	Re-run the golden set each quarter and after every model swap	Embedding model changes shift the optimal chunk size. Yesterday's tuning is not transferable.

What good looks like

> 0.90

Recall@20 on the golden set. Below this, no amount of prompt or reranker work will save the answers — the evidence is not reaching the model.

< 6

Chunks needed to answer a typical question. If it takes fifteen, your chunks are too small or your overlap is duplicating them.

1 day

Time to re-run the full evaluation after a config change. If it takes a week, the harness is the bottleneck, not the chunking.

Treat these as tripwires rather than targets. The value of the harness is not the numbers themselves — it is that a chunking argument in a design review becomes a fifteen-minute experiment instead of a matter of seniority.

Cheat sheet and rollout

The whole document compressed to one table, and a three-sprint path from wherever you are now to a configuration you can defend.

IF THIS IS TRUE OF YOUR CORPUS OR QUERIES...	...THEN START HERE
Clean headings, clauses or functions in every document	Structure-aware boundaries, size cap 800 tokens, zero overlap
No usable structure, ordinary prose	Recursive split, 500 tokens, 12% overlap
Answers are one or two sentences	200–400 token chunks plus parent-document expansion
Questions need synthesis across a whole document	Add a hierarchical summary level above the chunks
Chunks open with "it", "this", or an undefined term	Contextual prefixes, or late chunking if your provider supports it
Millions of documents, constant re-indexing	Deterministic boundaries only; reserve LLM passes for a curated subset
Small, static, high-stakes corpus	LLM or proposition chunking is affordable and usually wins
Answers must be cited or access-restricted	Never cross a structural or permission boundary; store offsets and ACL scope
Heavy tables, filings, numeric reports	Keep tables intact with repeated headers plus a generated caption; add entity and period context
Mixed sources in one system	Route by source type to per-type boundary policies and indexes
Fewer than about fifty documents	Consider skipping retrieval entirely and sending whole documents
You are not sure which of these applies	Ship the default, build the golden set, and let Section 07 decide

Three sprints

SPRINT 0

Baseline and instrumentation

Build the 50–100 query golden set. Stamp `chunk_config` and offsets on every chunk. Add a token-length assertion at index time. Measure `recall@20` on what you have today — that number is the argument-ender for the rest of the work.



SPRINT 1

The high-yield fixes

Hybrid dense + BM25 retrieval with a reranker over the top 20. Structure-aware boundaries for the source types that have structure. Parent-document assembly. Re-run the golden set and record the delta per change, not in aggregate.



SPRINT 2

Close the residual gap

Inspect the queries still failing. Orphan-reference failures point to contextual prefixes or late chunking. Synthesis failures point to a hierarchical summary layer. Table and figure failures point to extraction, not chunking. Fix by failure class, and re-run.

The honest summary

There is no best chunking strategy, and anyone offering you one has not seen your corpus.

There is, however, a defensible one. Match the boundary to the document's own structure. Match the size to the shape of the answers your users actually need. Add document context to chunks that cannot stand alone, and only to those. Keep the retrieval improvements — hybrid search and reranking — ahead of the chunking improvements in your queue, because that is the order the evidence supports. And hold on to an evaluation set that lets you change your mind cheaply when the corpus grows, the model changes, or the questions your users ask turn out to be different from the ones you designed for.

The teams that get this right are not the ones with the most sophisticated splitter. They are the ones who can answer, in one sentence and with a number, why their chunks are the size they are.

Continue the thread

RESEARCH COPILOT

Chronicle

Run the market, competitor and technical research behind decisions like this one — cited, auditable, and free with no signup.

deep-research.intelliforge.tech



FREE AI BOOTCAMP

IntelliForge Learning

Twelve weeks hands-on: build AI agents and production RAG pipelines, evaluate them properly, and ship your own micro-SaaS.

learning.intelliforge.tech

Ready for the full intensive? → upskill.intelliforge.tech



IF-RES

About this series

IntelliForge Research publishes applied engineering deep dives for teams shipping AI systems in production — decision guides, benchmark briefings and implementation playbooks, released as open documents.

intelliforge.tech/research

USE

Using this document

Share it, quote it, and take it into your own design review. If you run the Section 07 loop on your corpus and get a result that contradicts anything here, that result is worth more than this document — send it over.

founder@intelliforge.tech

IF-RES-2026-122 · Version 1.0 · Published 06/09/2026 · Girish Hiremath, Founder & Principal Engineer, IntelliForge Digital Services, Kondapur, Hyderabad

Sources referenced: Anthropic, "Contextual Retrieval in AI Systems" (2024) · Günther et al., "Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models" (arXiv:2409.04701) · Qu, Tu & Bao, "Is Semantic Chunking Worth the Computational Cost?" (Findings of NAACL 2025).

© 2026 IntelliForge Digital Services · intelliforge.tech/research